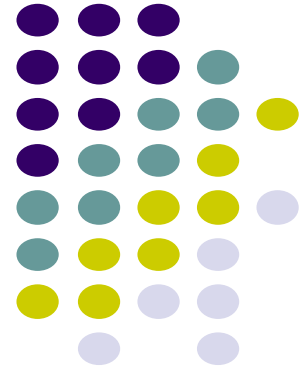


ErSE222: Machine learning in Geoscience

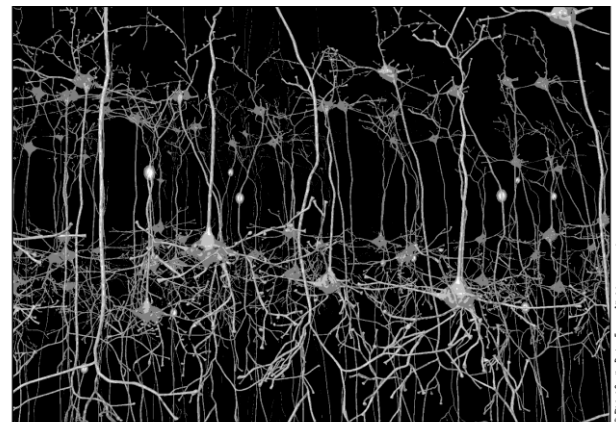
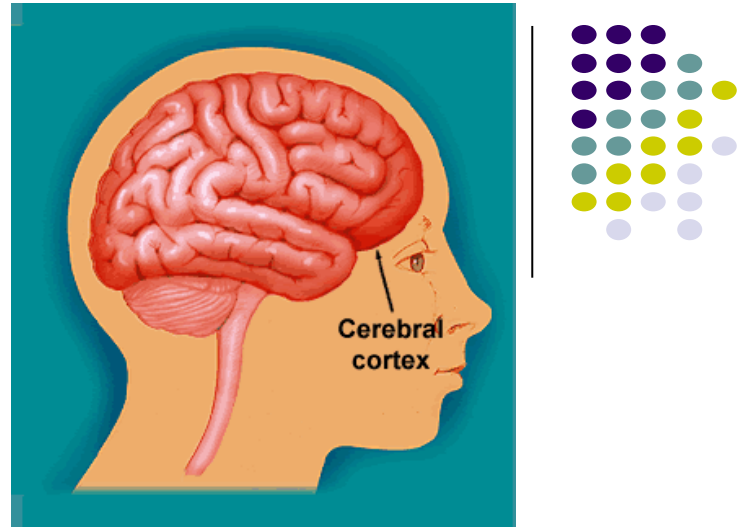
Feb. 8th, 2026

Tariq Alkhalifah and Omar Saad



Neural Network

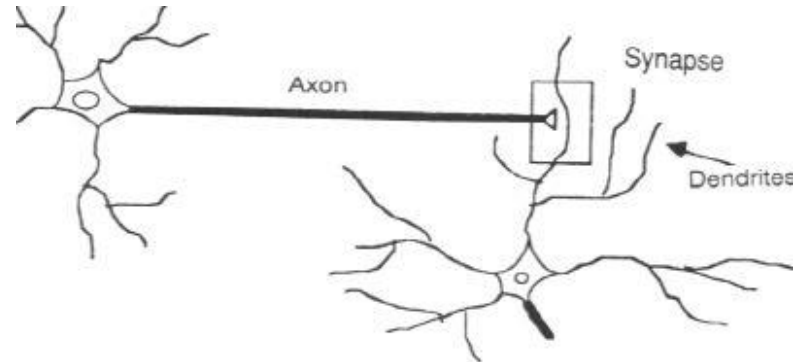
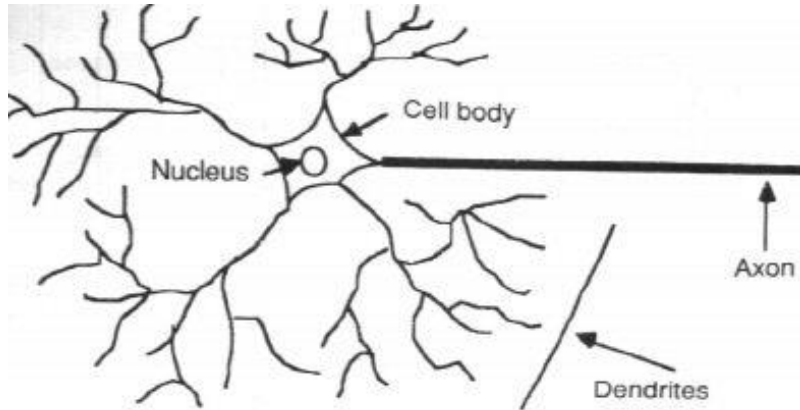
- **Neuron:** the fundamental computational units
- **Synapses:** the connections between neurons
- **Layer:** neurons are organized into layers
- ***Extremely complex:*** around 10^{11} neurons in the brain, each with 10^4 connections



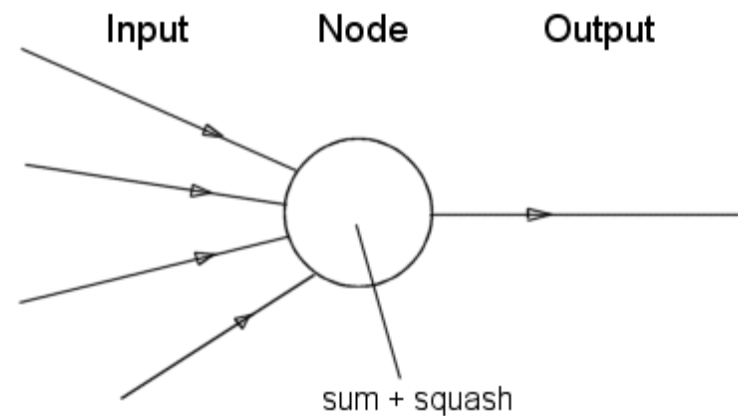
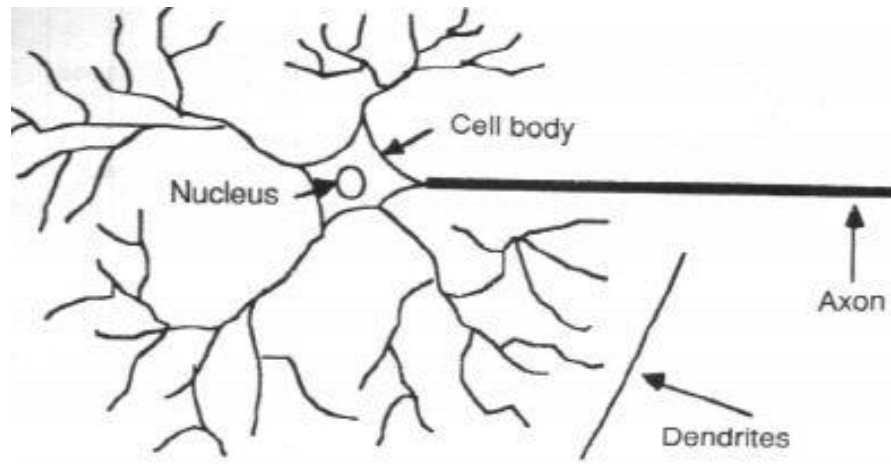
The Neuron - A Biological Information Processor



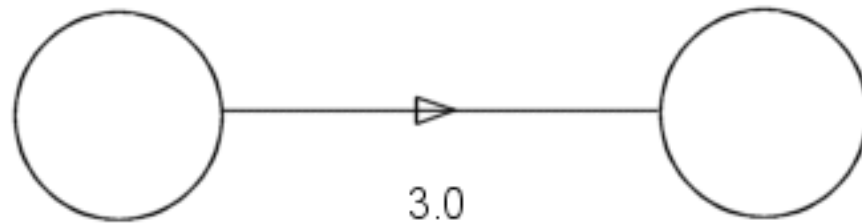
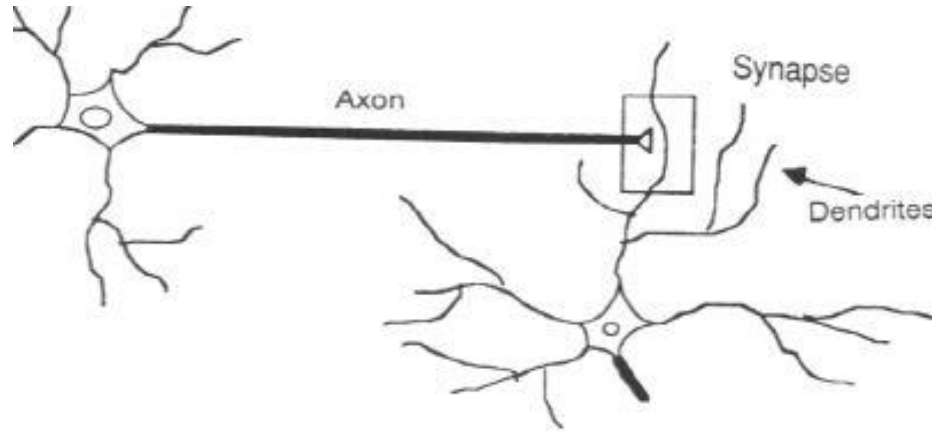
- *dendrites* - the receivers
- *soma* - neuron cell body (sums input signals)
- *axon* - the transmitter
- *synapse* - point of transmission



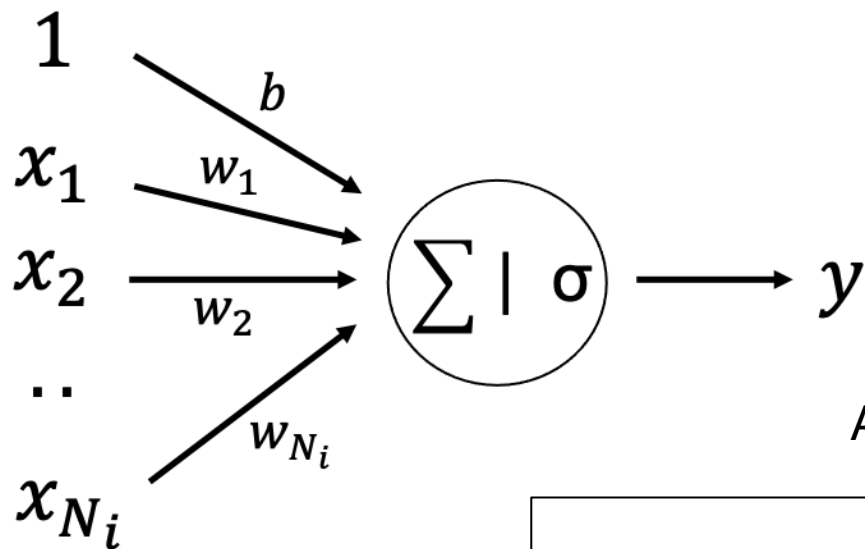
Neural Network



Neural Network



Perceptron



Activation function Weights Bias

$$y = \sigma\left(\sum_i w_i x_i + b\right) = \sigma\left(\sum_i \mathbf{w}^T \mathbf{x} + b\right)$$

Perceptron

$$X_1 = -0.06$$

$$W_1 = 2.7$$

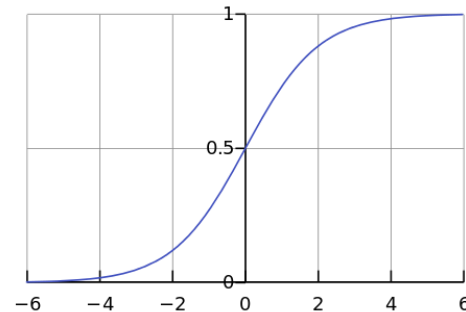
$$X_2 = -2.5$$

$$W_2 = 2.7$$

$$X_3 = 1.4$$

$$W_3 = 0.002$$

$$z = -0.06 \times 2.7 + 2.5 \times 8.6 + 1.4 \times 0.002 = 21.34$$

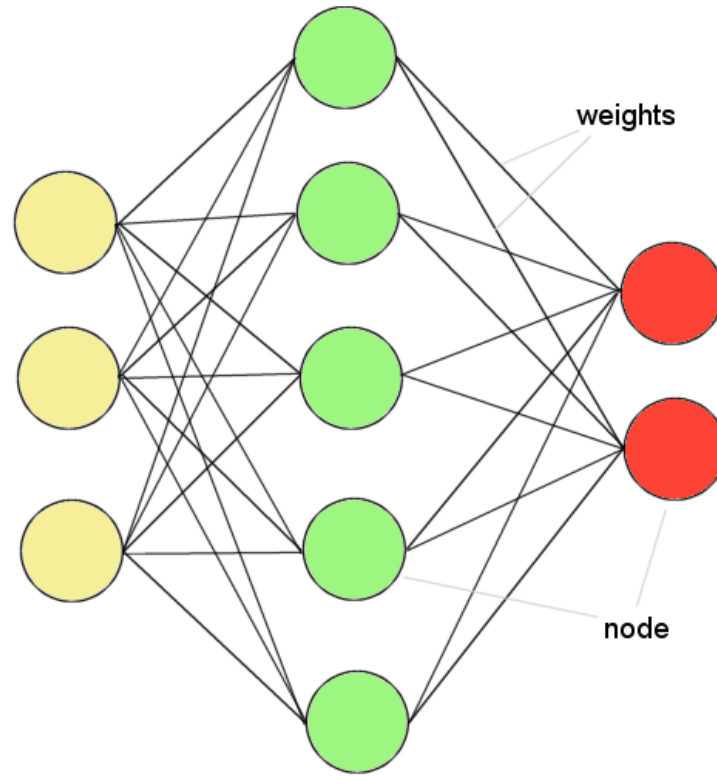


MLP

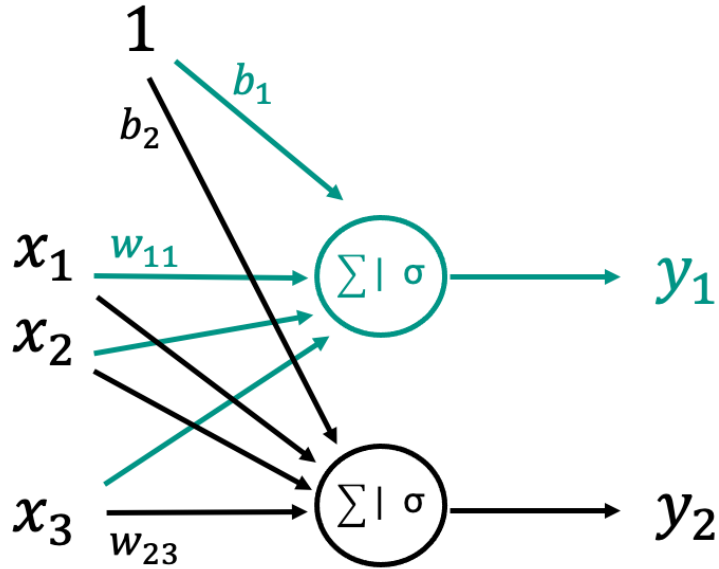
Input

Hidden

Output



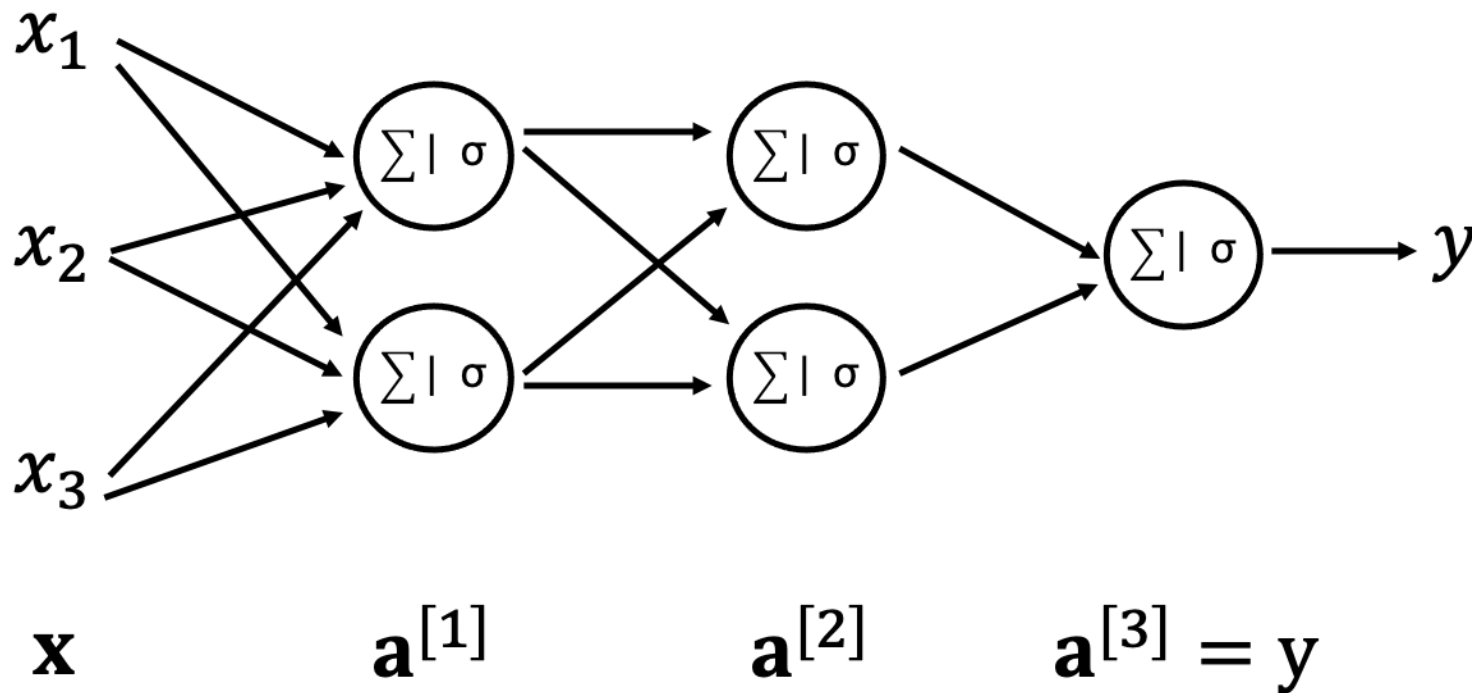
MLP



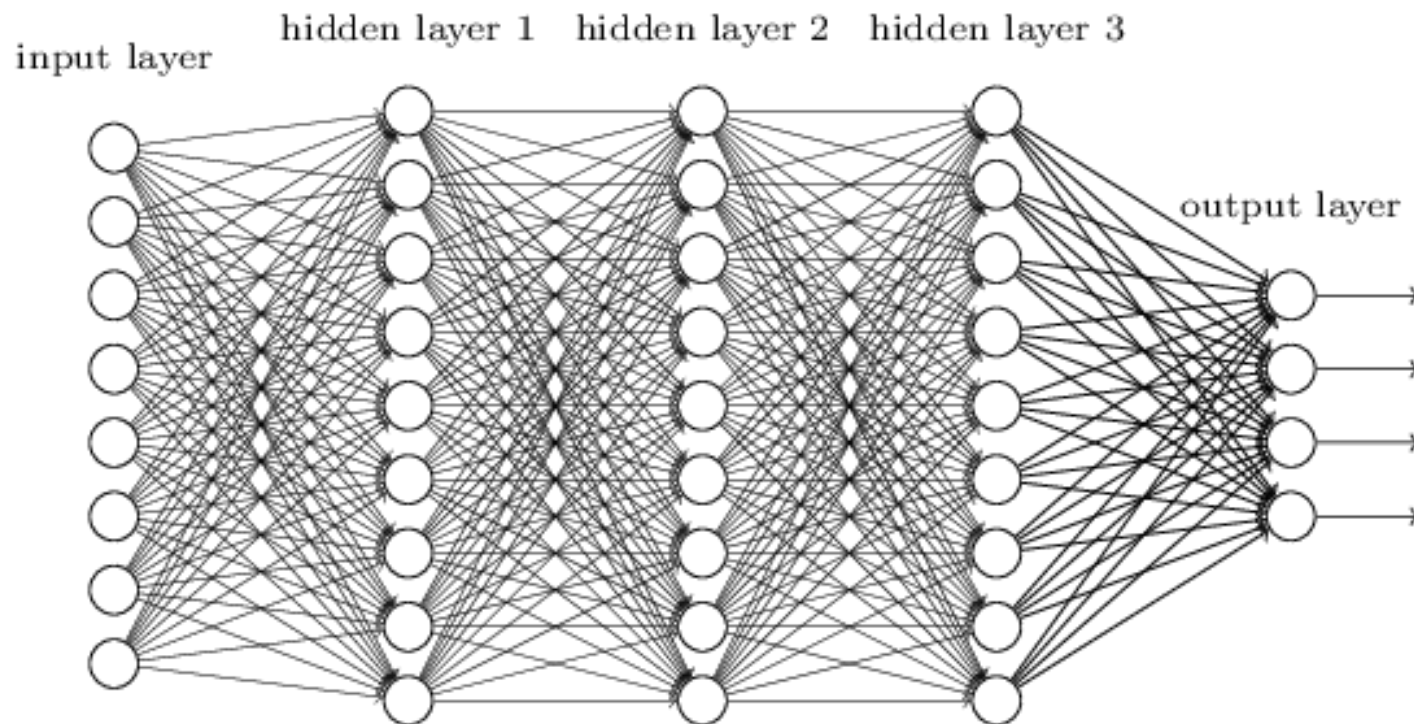
$$\mathbf{W} \in \mathbb{R}^{N_o \times N_i} \quad \mathbf{b} \in \mathbb{R}^{N_o}$$

$$y_j = \sigma\left(\sum_i w_{ji}x_i + b\right), \quad \mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

Deep Network (3-layers)



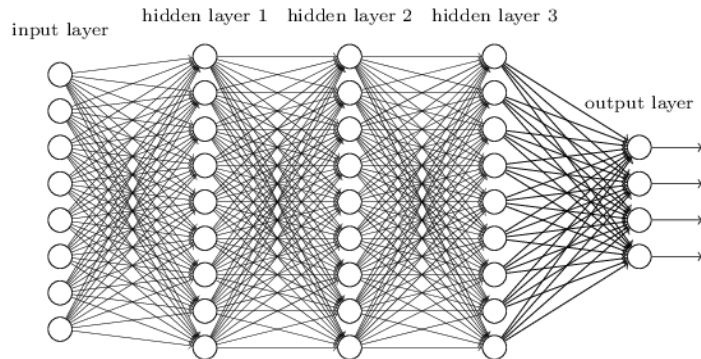
Deep Network (3-layers)



Deep Network (3-layers)



- *Input layer*: first layer taking the input vector $\mathbf{x}$ as input and returning an intermediate representation $\mathbf{z}^{[1]}$;
- *Hidden layers*: second to penultimate layers taking as input the previous representation $\mathbf{z}^{[i-1]}$ and returning a new representation $\mathbf{z}^{[i]}$;
- *Output layer*: last layer producing the output of the network $\mathbf{y}$;
- *Depth*: number of hidden layers (plus output layer);
- *Width*: number of units in each hidden layer.



Activation Functions - motivation



The need for nonlinearities comes from the recognition that:

$$\mathbf{y} = \sigma(\mathbf{W}^{[3]}\sigma(\mathbf{W}^{[2]}\sigma(\mathbf{W}^{[1]}\mathbf{x}))) = \mathbf{W}^{[3]}\mathbf{W}^{[2]}\mathbf{W}^{[1]}\mathbf{x} = \mathbf{W}\mathbf{x}$$

$$\sigma(\mathbf{x}) = \mathbf{I}\mathbf{x} = \mathbf{x},$$

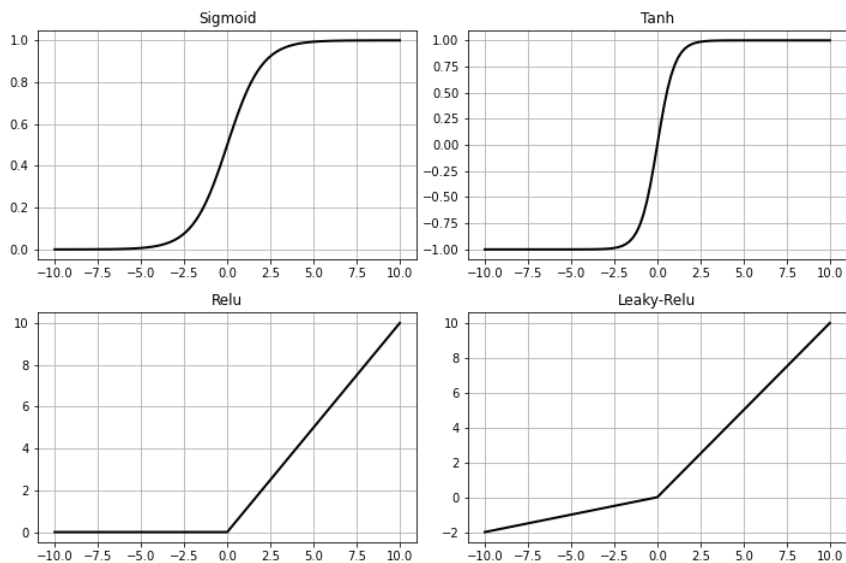


One single linear model!

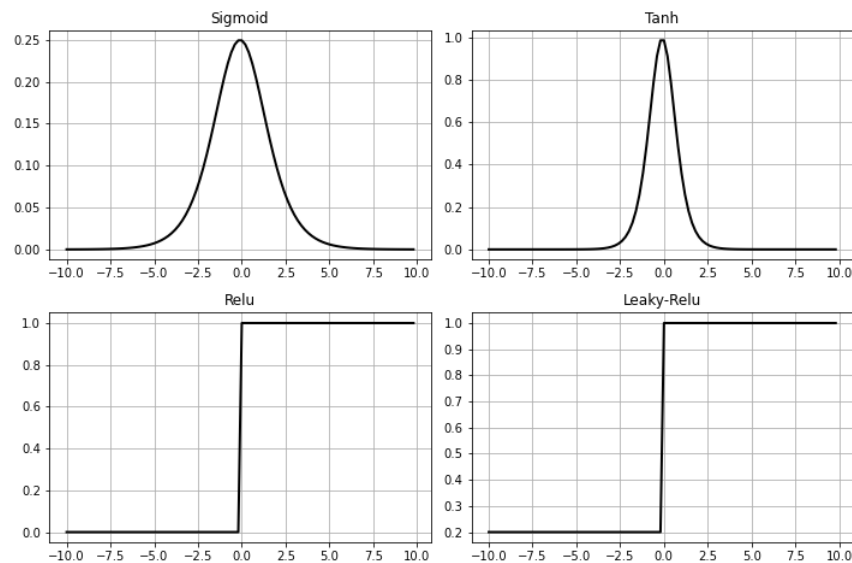
Activation Functions



Activation functions



Activation function derivatives

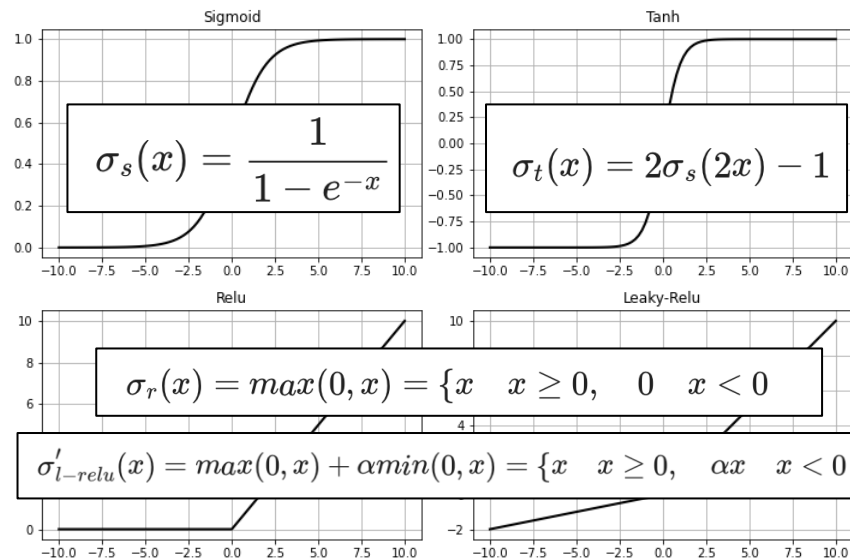


Activation Functions

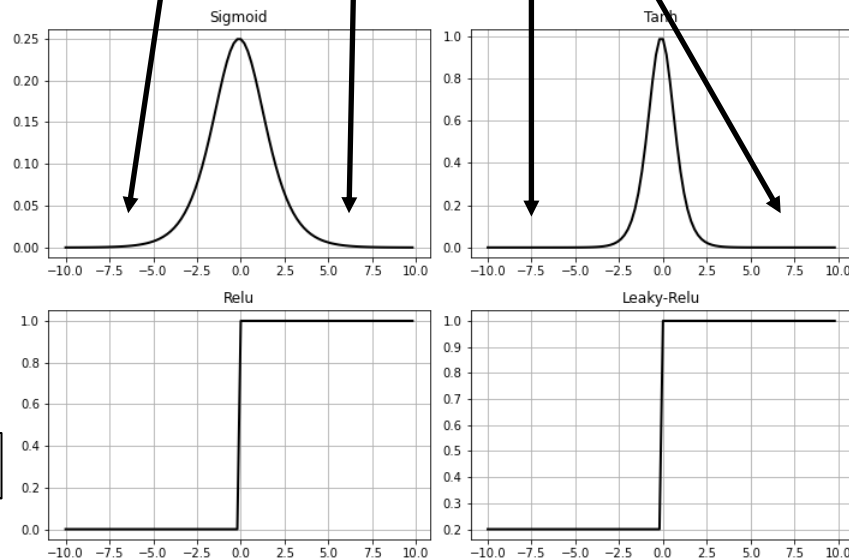


Differentiable, but saturate away from 0

Activation functions



Activation function derivatives



Non-differentiable, but does not saturate



Learning algorithms

- Dataset: $\mathbf{x} = [x_1, x_2, \dots, x_{N_f}]^T$ $\mathbf{X} = [\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N_s)}]$ Inputs / features

$\mathbf{y} = [y_1, y_2, \dots, y_{N_t}]^T$ $\mathbf{Y} = [\mathbf{y}^{(1)}, \mathbf{y}^{(2)}, \dots, \mathbf{y}^{(N_s)}]$ Outputs / labels

- Model: $\mathbf{y} = f_{\theta}(\mathbf{x})$

- Cost function: $J_{\theta} = \frac{1}{N_s} \sum_{j=1}^{N_s} \mathcal{L}(\mathbf{y}^{(j)}, f_{\theta}(\mathbf{x}^{(j)}))$

- Optimization algorithm: $\hat{\theta} = \underset{\theta}{\operatorname{argmin}} J_{\theta}$





Loss Function (Regression)

The most commonly used cost function is:

$$J_{\theta} = \underset{\substack{\uparrow \\ \text{Mean squared error}}}{MSE}(\mathbf{y}_{train}, \hat{\mathbf{y}}_{train}) = \frac{1}{N_s} \|\mathbf{y}_{train} - \hat{\mathbf{y}}_{train}\|_2^2 = \frac{1}{N_s} \sum_i^{N_s} \underbrace{(y_{train}^{(i)} - \hat{y}_{train}^{(i)})^2}_{\mathcal{L}}$$

which is minimized as follows:

$$\hat{\theta} = \min_{\theta} J_{\theta} \rightarrow \theta_{i+1} = \theta_i - \alpha \nabla J_{\theta}$$

Once the model is trained (= best parameters are learned) one can estimate the solution from a new input – INFERENCE

$$y_{test} = \tilde{\mathbf{x}}_{test}^T \hat{\theta}$$

Loss Function (Classification)



The loss function is called **binary cross-entropy**

$$\mathcal{L}(y_{train}^{(i)}, \hat{y}_{train}^{(i)}) = \underbrace{-(y_{train}^{(i)} \log(\hat{y}_{train}^{(i)}))}_{\text{for 'true' labels (y=1)}} + \underbrace{(1 - y_{train}^{(i)}) \log(1 - \hat{y}_{train}^{(i)})}_{\text{for 'false' labels (y=0)}}$$

and the total cost function is:

$$J_{\theta} = \frac{1}{N_s} \sum_i^{N_s} \mathcal{L}(y_{train}^{(i)}, \hat{y}_{train}^{(i)}) \longrightarrow \hat{\theta} = \min_{\theta} J_{\theta} \rightarrow \theta_{i+1} = \theta_i - \alpha \nabla J_{\theta}$$

Sigmoid Activation Function

$$\sigma_s(x) = \frac{1}{1 + e^{-x}}$$



Loss Function (Classification)

Categorical cross-entropy loss in multi-class classification tasks;

$$CE = - \sum_{i=1}^{i=N} y_i \cdot \log(\hat{y}_i)$$

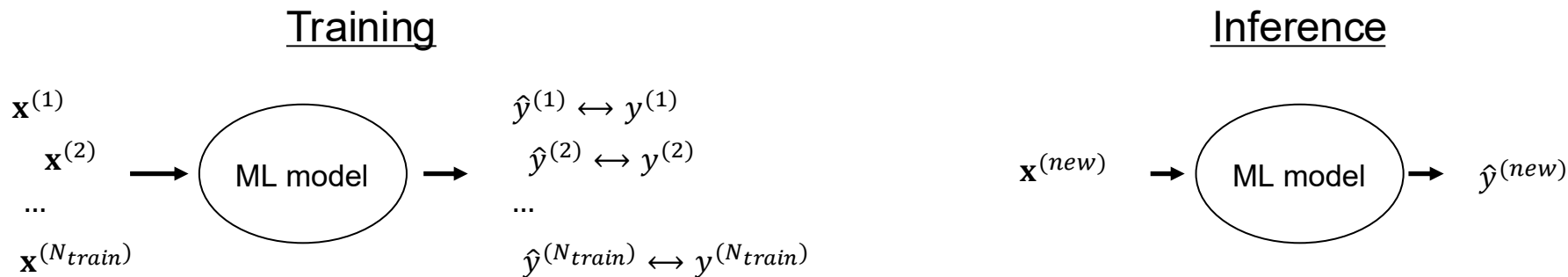
$$CE = -[y_1 \cdot \log(\hat{y}_1) + y_2 \cdot \log(\hat{y}_2) + y_3 \cdot \log(\hat{y}_3)]$$

Softmax Layer

Ultimately goal of a ML model (Supervised Learning)



A model is useful if it can ***perform well on new, previously unseen data***. This property of a model is also generally referred to as ***generalization***.



ML data 'structuring' (Supervised Learning)



- Training dataset: $\{\mathbf{X}_{train}, \mathbf{Y}_{train}\}$, used to train the model (e.g., learn the free-parameters θ of a NN);
- Validation dataset: $\{\mathbf{X}_{valid}, \mathbf{Y}_{valid}\}$, used to select the hyperparameters of the model;
- Testing dataset: $\{\mathbf{X}_{test}, \mathbf{Y}_{test}\}$, used only once a model is finalized (trained and optimized) to produce an *unbiased* estimate of model performance.

ML performance



Different measures for different datasets:

- **Training error** (or performance): overall error (or performance) computed over the training dataset;
- **Validation error** (or performance): overall error (or performance) computed over the validation dataset.
- **Test/Generalization error** (or performance): overall error (or performance) computed over the testing dataset.

Classification evaluation metrics (Confusion Matrix)



		True Labels	
		Positive (P)	Negative (N)
Predicted Labels	Positive	Correct / True Positive (TP)	Type 1 Error / False Positive (FP)
	Negative	Type 2 Error / False Negative (FN)	Correct / True Negative (TN)



Classification evaluation metrics

- Precision

$$Pr = \frac{TP}{TP+FP}$$

Appropriate when minimizing false positives is the focus..

- Recall:

$$Rc = \frac{TP}{TP+FN} = \frac{TP}{P}$$

Appropriate when minimizing false negatives is the focus..

- Accuracy:

$$Ac = \frac{TP+TN}{TP+TN+FP+FN} = \frac{TP+TN}{P+N}$$

Percentage of correct predictions over the total number of cases. Combines both error types (in the denominator), it is therefore a more global measure of the quality of the model.

- F1-score:

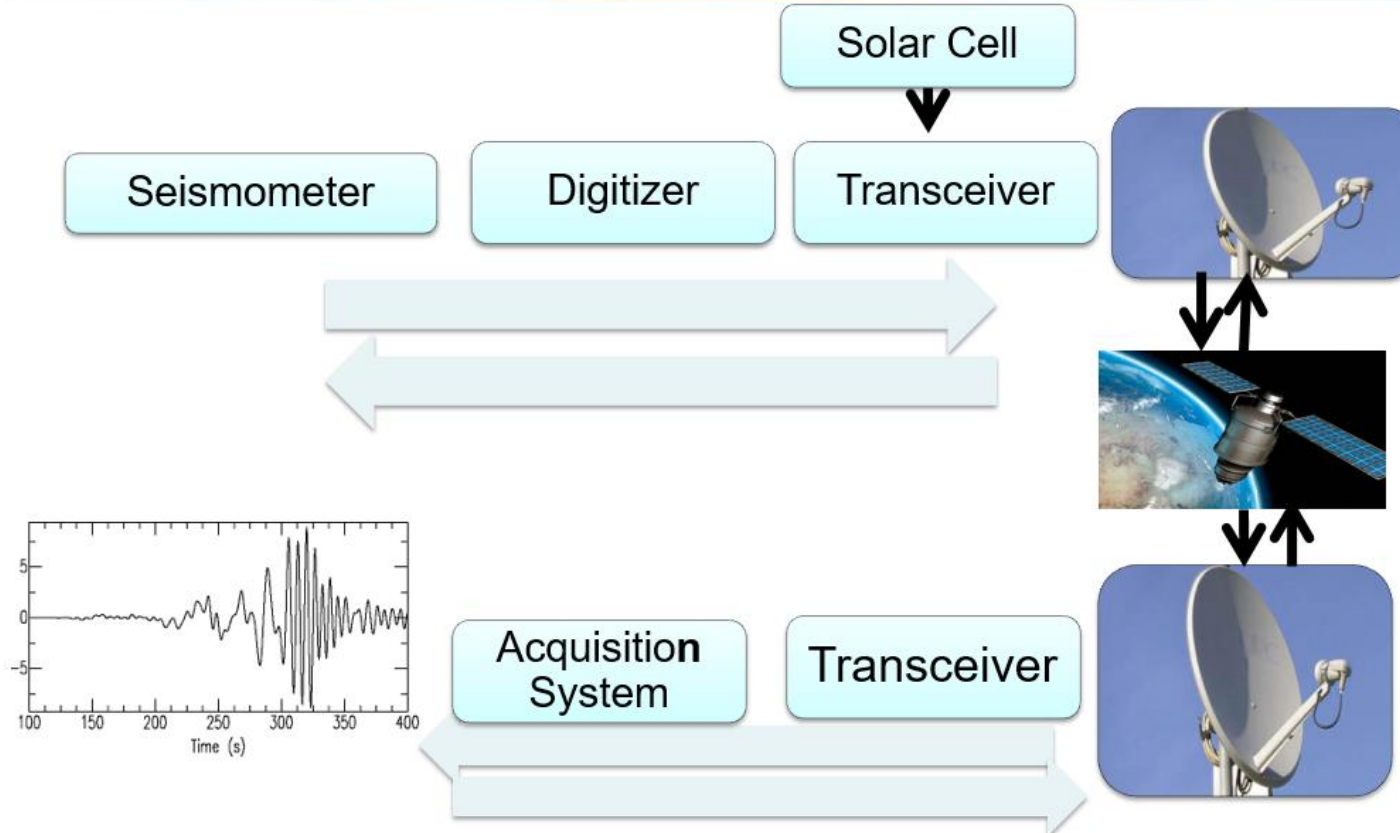
$$2 \frac{Pr \cdot Rc}{Pr + Rc}$$

Combines precision and recall into a single measure

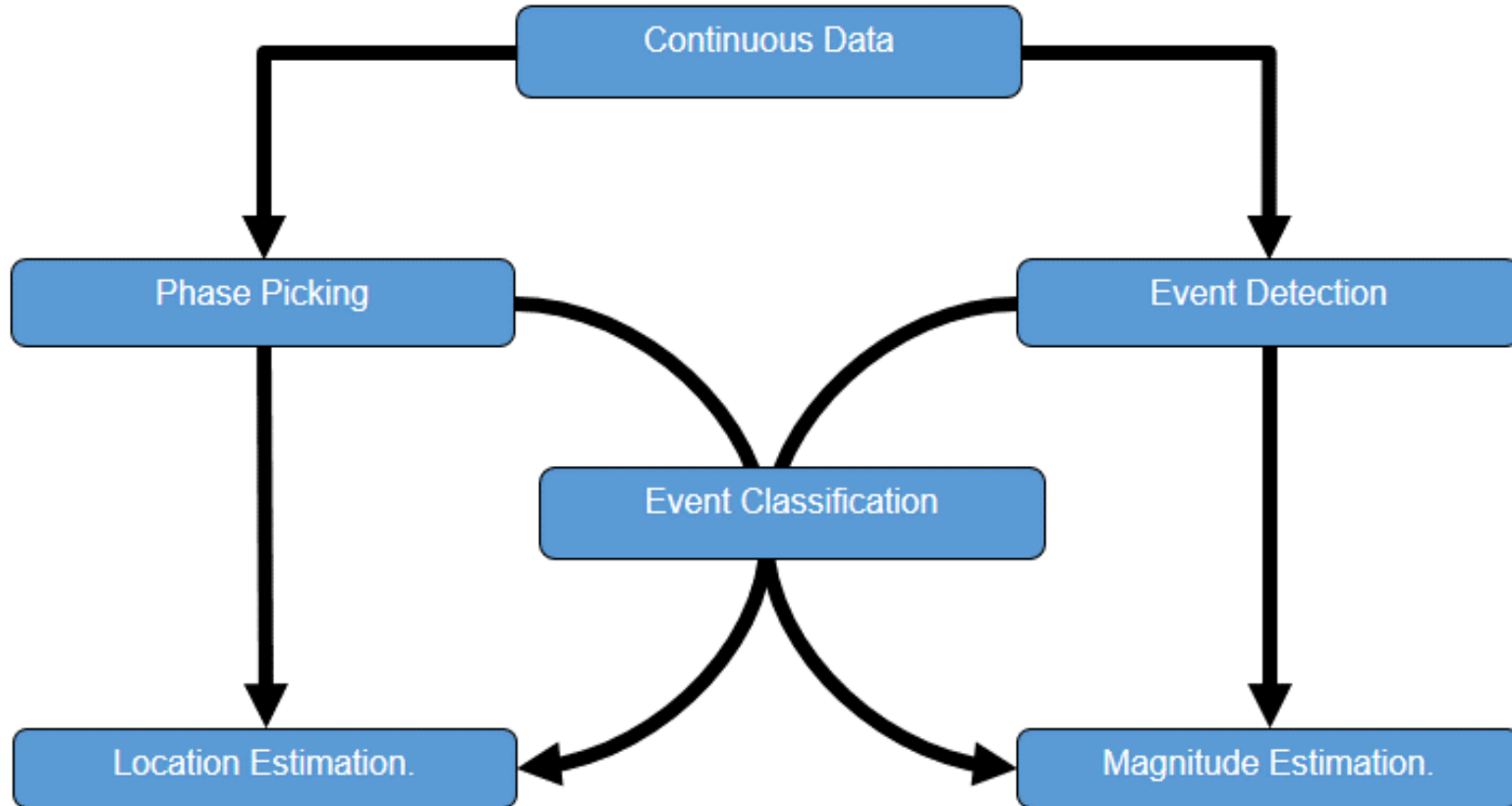


Application

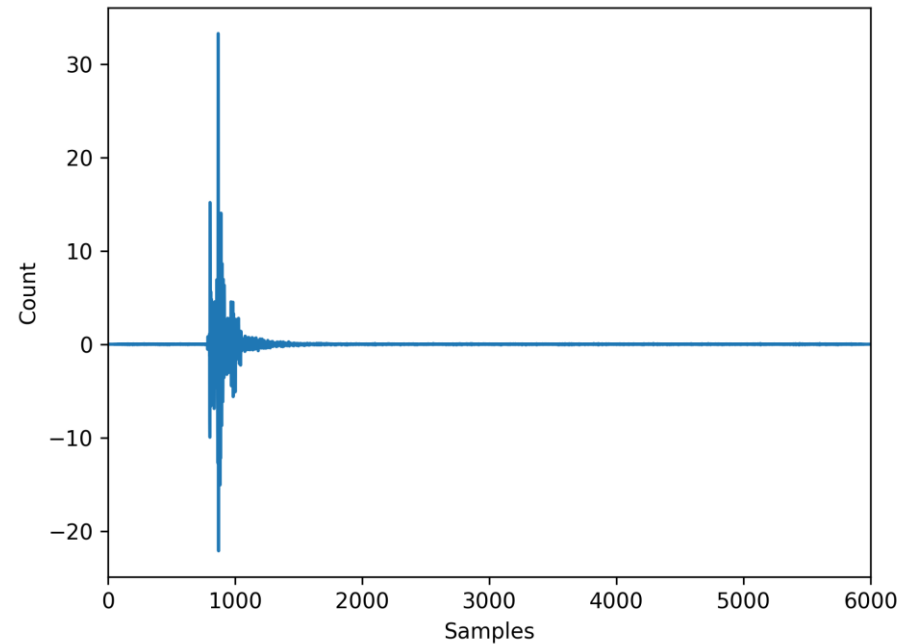
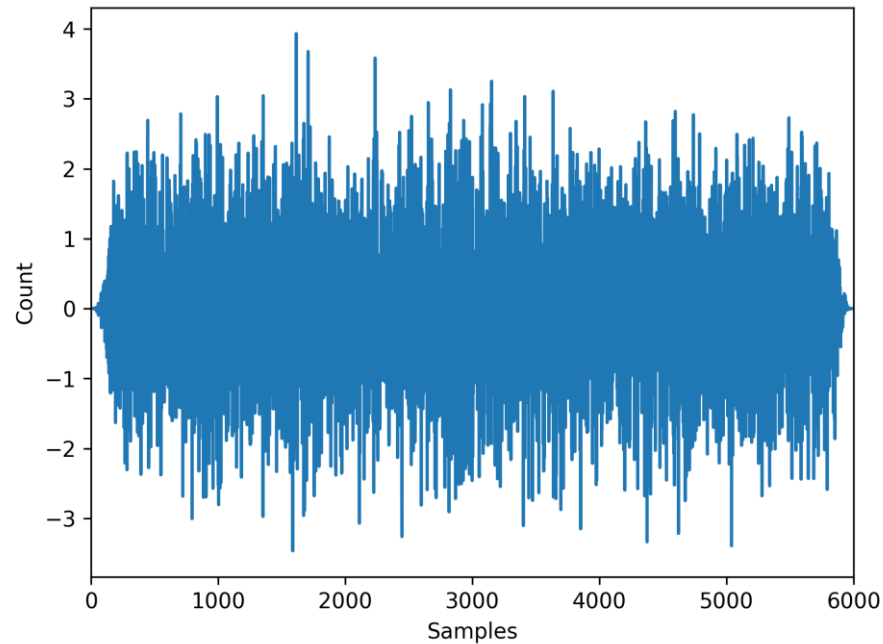
Earthquake Monitoring System



Earthquake Monitoring System



Seismic Event Classification



A wide-angle photograph of a desert landscape. In the foreground, there are prominent, layered rock formations, likely sandstone, with a warm, golden-brown hue. The rocks show clear horizontal strata. Beyond the rocks, a vast, flat valley stretches out, with some distant, low-lying hills and a few small structures visible on the horizon. The sky is a pale, hazy blue. The overall scene is serene and expansive.

Thank You

30 17:50